ONESIM

Bulk Order Service

Mechanism, Algorithm & Implementation Guide

ONETECH PRO LLC | v1.0 | March 2026 | Confidential

1. What This Service Does

The Bulk Order Service is the backend engine that sits between the partner placing an order and the end customer receiving a working eSIM. It takes a confirmed group order, orchestrates N sequential eSIM purchases from the eSIMAccess API, generates N unique tokenized delivery links, and dispatches each one to the correct customer via their chosen delivery method — SMS, email, or prepares them for manual distribution.

It is NOT a single API call. It is a multi-stage pipeline that must be resilient, logged, and independently restartable at each stage.

Scope of this document

Covers: Group bulk orders (B2B partner flow) — the primary use case.

Also applies: Single-customer partner orders (simpler version of the same pipeline).

Skipped: Click.uz payment handling — placeholder exists in the flow, implementation deferred.

Skipped: Detailed SMS/email provider API — referenced only at the dispatch step.

2. Relevant Database Tables

Your database already has all the required tables. This section maps each table to its role in the pipeline so the service is built against reality, not theory.

Table	Role in Bulk Order Pipeline
group_orders	The master record for a bulk order. Tracks the overall status of the entire batch. One row per group purchase action.
customer_groups	Defines the group — destination, travel dates, default delivery method, default package.
customer_group_members	Join table: which partner_customers belong to this group.
partner_customers	Individual end-customer records — name, phone, email, delivery preferences.
orders	One row per individual eSIM provisioned. Each member of the group gets their own orders row after eSIMAccess responds. Contains: iccid, qr_code_data, activation_code, short_url, delivery_method, delivery_status.
esim_delivery_logs	Append-only log of every delivery attempt per order. Stores provider_message_id from Eskiz/email, status, attempt_count, failure_reason.
partner_earnings	One row per order recording the discount saving. Written after each successful eSIM provisioning.
payments	Placeholder for Click.uz payment. The group_order is linked here. For MVP: this row is created but payment_status is set manually / skipped.
payment_audit_log	Append-only audit trail. Every state change, every API call response, every error should write here.

Key field to note: orders.short_url

This column already exists in your DB. It will hold the tokenized delivery link for each customer.
Format: onesim.uz/e/{8-char-token} — generated during the pipeline, before delivery dispatch.
The token maps to this specific orders row. One token = one customer = one eSIM.

3. The group_orders Status Machine

Everything in the pipeline is driven by the status field on group_orders. Every stage transition must be atomic — update the status and log it in the same operation. This ensures the service is safely restartable.

State	Set By	Meaning
draft	Frontend	Partner has created the order intent but not confirmed. Package selected, members known.
plan_selected	Frontend	Partner has reviewed summary and pressed 'Pay & Confirm'. Pre-payment checkpoint.
payment_pending	Service / Click	Payment initiated. Awaiting Click.uz callback. (MVP: skip directly to 'paid')
paid	Service / Click	Payment confirmed. Pipeline is authorized to call eSIMAccess. This is the trigger for the service to start.
api_ordering	Service	Service is actively calling eSIMAccess API one-by-one for each member. In-progress.
completed	Service	All eSIMs provisioned successfully. Delivery dispatched (or manual PDF ready). Final happy state.
partial	Service	Some eSIMs provisioned, some failed. Partner must be notified. Manual intervention needed for failed ones.
failed	Service	All eSIM provisioning failed (or payment failed). No eSIMs delivered. Refund process triggered.

MVP shortcut: skipping payment

For MVP, there is no Click.uz integration yet. The frontend will call a Supabase Edge Function directly after partner confirms.

The Edge Function sets group_orders.status = 'paid' immediately and proceeds.

The payments table row is created but payment_status = 'SKIPPED_MVP' for traceability.

When Click.uz is integrated later, the webhook sets status = 'paid' and the same pipeline continues from there — zero changes needed downstream.

4. Full Pipeline — Stage by Stage

Stage 0: Pre-flight Checks (before touching any APIs)

Before the service does anything expensive, it must validate the order is in a safe state to proceed. If any check fails, abort and log — do not proceed.

#	Step / Action	Detail
0.1	**Load group_order**	Fetch group_orders row by ID. Confirm status = 'paid'. If not 'paid', abort — another process may already be handling it.
0.2	**Idempotency check**	Check if any orders rows already exist with this group_order_id. If yes, this is a retry — skip already-provisioned members, continue only failed ones.
0.3	**Load members**	JOIN customer_group_members → partner_customers for this group. Build the member list. Each member needs: id, full_name, phone, email, delivery_method.
0.4	**Validate member data**	For each member: if delivery_method = 'sms' → must have valid phone. If 'email' → must have valid email. Flag invalid members — they will be skipped with status 'DELIVERY_BLOCKED', eSIM still provisioned.
0.5	**Load package details**	Fetch package from esim_packages by group_orders.package_code. Confirm is_active = true. Snapshot the price fields for the orders rows.
0.6	**Load partner details**	Fetch partner record for discount_rate. Calculate: wholesale_price = retail_price × (1 - discount_rate/100). Snapshot for partner_earnings.
0.7	**Set status → api_ordering**	UPDATE group_orders SET status = 'api_ordering', ordered_at = now(). Log to payment_audit_log: event = 'BULK_ORDER_STARTED', member count, package_code.

Stage 1: eSIM Provisioning Loop

For each member in the list, call eSIMAccess API to order one eSIM. This is a sequential loop — not parallel — to avoid rate limits and to make partial failures traceable.

#	Step / Action	Detail
1.1	**For each member (loop)**	Iterate through the validated member list one by one.
1.2	**Check if already done**	Query: does an orders row exist for this group_order_id AND end_customer_id? If yes and status ≠ 'FAILED' → skip this member (idempotency).
1.3	**Call eSIMAccess API**	POST to eSIMAccess /esims/order endpoint with package_code and quantity=1. See eSIMAccess API cheatsheet for exact payload. Store raw API response.

1.4	**On API success**	Extract: iccid, qr_code_data (base64 or URL), smdp_address, activation_code, esim_tran_no. Proceed to step 1.5.
1.5	**Generate token**	Use nanoid(8) — URL-safe alphanumeric, no ambiguous characters. Check uniqueness against orders.short_url (collision probability is negligible but verify). Build link: onesim.uz/e/{token}.
1.6	**Write orders row**	INSERT into orders: user_id = partner's user_id, partner_id, group_order_id, end_customer_id, package_code, iccid, qr_code_data, activation_code, smdp_address, short_url = token link, delivery_method = member's method, order_status = 'ALLOCATED', source_type = 'b2b_partner', discount fields.
1.7	**Write partner_earnings**	INSERT into partner_earnings: retail_price, partner_paid, discount_amount — one row per eSIM provisioned.
1.8	**On API failure**	INSERT orders row with order_status = 'FAILED', error_message = API error. Log to payment_audit_log. Continue loop — do NOT abort entire batch.
1.9	**Rate limit delay**	After each API call, sleep 100–200ms to respect eSIMAccess rate limits. For batches > 20, consider 300ms.

Why sequential, not parallel?

Parallel calls risk hitting eSIMAccess rate limits, causing partial failures that are hard to attribute.
Sequential makes each failure clearly attributable to a specific member.
For typical group sizes (5–30 people), sequential at 200ms delay = 1–6 seconds total. Acceptable.
For large batches (50+), consider chunked parallel: groups of 5 with delay between chunks.

Stage 2: Post-Provisioning Evaluation

After the loop completes, evaluate the outcome before dispatching delivery.

#	Step / Action	Detail
2.1	**Count outcomes**	Query: SELECT order_status, COUNT(*) FROM orders WHERE group_order_id = X GROUP BY order_status. Get: total, succeeded (ALLOCATED), failed.
2.2	**Determine final status**	If failed = 0 → 'completed'. If failed > 0 AND succeeded > 0 → 'partial'. If succeeded = 0 → 'failed'.
2.3	**Update group_orders**	SET status = computed status, esimaccess_batch_ref = first transaction ID (for reference), updated_at = now().
2.4	**Update partner stats**	UPDATE partners: total_orders += 1, total_spent += partner_paid_total, total_savings += discount_total, last_order_at = now().
2.5	**Log evaluation**	Write to payment_audit_log: event = 'PROVISIONING_COMPLETE', succeeded count, failed count, final status.

2.6	**If all failed**	Trigger refund flag — set a metadata field indicating refund review needed. Notify ONESIM admin. STOP — do not proceed to delivery.

Stage 3: Delivery Token Page Preparation

Before dispatching any notifications, the landing page must be ready to serve. Since we are using onesim.uz/e/{token} as the URL, the data the page needs must be readable from the orders row.

The landing page (built on the main onesim.uz site) simply does:

1. Receive token from URL path
2. Query: SELECT * FROM orders WHERE short_url LIKE '%{token}' — or better: store just the token in a dedicated column
3. Render: QR code (generated client-side from qr_code_data / activation_code), package info, partner name, install guide, support link

Recommended: add esim_tokens table

Rather than embedding the token inside the full URL in short_url, consider a dedicated esim_tokens table:

token (PK, 8 chars) | order_id (FK) | expires_at | first_viewed_at | view_count

Benefits: faster lookup by token, expiry control, view analytics, clean invalidation on resend.

Your orders.short_url then stores the full URL: onesim.uz/e/{token} for display purposes.

Stage 4: Delivery Dispatch Loop

Now dispatch the actual notification to each customer whose eSIM was successfully provisioned. This is a second loop over ALLOCATED orders only.

#	Step / Action	Detail
4.1	**Fetch ALLOCATED orders**	SELECT orders JOIN partner_customers WHERE group_order_id = X AND order_status = 'ALLOCATED'. Separate into three queues: sms_queue[], email_queue[], manual_queue[].
4.2	**Pre-flight: Eskiz auth**	If sms_queue is non-empty: call Eskiz POST /api/auth/login with stored email + secret_key credentials. Receive bearer token (TTL 30 days). Store token in function memory for this run. Do NOT store in DB — treat as ephemeral per-invocation.
4.3	**Pre-flight: Eskiz balance**	GET /api/user/get-limit with bearer token. Parse response.data.balance. If balance = 0 or insufficient: mark all SMS orders as DELIVERY_BLOCKED_NO_BALANCE, log event, alert admin, skip SMS queue entirely. Continue to email/manual queues.
4.4	**Build SMS batch payload**	For each order in sms_queue, compose: { user_sms_id: order.id, to: partner_customer.phone (numeric, no +), text: sms_body }. Phone format: 998XXXXXXXXX as integer. SMS body — see template below.
4.5	**Send SMS batch**	POST /api/message/sms/send-batch with Authorization: Bearer {token}. Body: { messages: [...], from: '4546', dispatch_id: numeric_hash_of_group_order_id, callback_url:

		'https://api.onesim.uz/functions/v1/eskiz-callback' }. One call for the entire group.
4.6	**Process SMS batch response**	Eskiz returns: { id: request_uuid, status: ['waiting','waiting',...] }. The top-level id is the batch request_id. Store it. Write esim_delivery_logs row per order: provider_message_id = request_uuid, status = 'sent', method = 'sms'.
4.7	**Email queue**	For each order in email_queue: compose HTML email (QR code embedded, activation code, install guide, partner name). Call email provider (SendGrid/Resend). Store returned message_id in esim_delivery_logs.
4.8	**Manual queue**	No external call. Mark delivery_status = 'manual_pending' on each order. Flag group_order metadata: has_manual_delivery = true. PDF will be generated on partner request.
4.9	**On any delivery failure**	Log to esim_delivery_logs: status = 'failed', failure_reason = error message. Increment attempt_count. Do NOT retry immediately. Continue loop — one failure never blocks others.

SMS Message Template

Exact content sent to end customer via Eskiz:

Template:

"{CompanyName}: eSIM tayyor! {Country} {DataGB}GB. O'rnatish: onesim.uz/e/{token}"

Live example:

"Dubay Safari: eSIM tayyor! UAE 5GB. O'rnatish: onesim.uz/e/aB3kR9xQ"

Character count:

~65 chars = 1 SMS part = cheapest possible. No Cyrillic = GSM 0338 encoding = max 160 chars/part.

Eskiz sender ID:

from: '4546' (Eskiz default numeric sender — no custom nickname needed for MVP)

Eskiz Delivery Status Tracking — Callback Flow

Eskiz supports an optional callback_url that fires a POST when delivery status changes. This is the proper way to get real delivery receipts rather than polling.

#	Step / Action	Detail
CB.1	**Register callback URL**	Include callback_url in every send-batch call: https://api.onesim.uz/functions/v1/eskiz-callback (a dedicated public Edge Function).
CB.2	**Eskiz fires callback**	When each SMS part is delivered (or fails), Eskiz POSTs: { request_id, message_id, user_sms_id, country, phone_number, sms_count, status, status_date }. user_sms_id = our orders.id.
CB.3	**Callback function receives**	Look up esim_delivery_logs by provider_message_id = request_id AND order_id = user_sms_id.

C B. 4	**Map Eskiz status**	DELIVRD → 'delivered'. UNDELIV / REJECTD → 'failed'. ACCEPTED / ENROUTE → 'sent' (no change yet). DELETED → 'failed' with reason 'invalid_sender'.
C B. 5	**Update delivery log**	UPDATE esim_delivery_logs: status = mapped_status, delivered_at / failed_at = status_date, failure_reason if applicable.
C B. 6	**Update order JSONB**	UPDATE orders.delivery_status JSONB with final status for fast querying on partner dashboard.

Eskiz credentials — stored as Supabase secrets

Email: yoqub.xamidoff19@gmail.com

Secret key: stored as ESKIZ_SECRET_KEY in Supabase Edge Function secrets (never hardcoded).

Token refresh: Eskiz token TTL is 30 days. On each function invocation, call /api/auth/login fresh.

Alternative: cache the token in a DB secrets table with expires_at. Refresh only when expired.

PATCH /api/auth/refresh can extend an existing token without re-login — useful for long-running jobs.

Stage 5: Final Summary & Notifications

#	Step / Action	Detail
5. 1	**Compute delivery summary**	Count: delivery sent, delivery failed, manual pending. Build summary object.
5. 2	**Update group_orders.metadata**	Store delivery summary: { sms_sent: N, email_sent: N, manual: N, delivery_failed: N, completed_at: timestamp }.
5. 3	**Notify partner**	Send in-app notification (or email to partner's business_email): 'Your order for {group_name} is complete. {N} eSIMs delivered, {X} pending manual distribution.' Include link to order detail page.
5. 4	**Log completion**	Write final event to payment_audit_log: event = 'BULK_ORDER_COMPLETED', full summary.
5. 5	**Generate manual PDF**	If any orders have delivery_method = 'manual': trigger PDF generation job. PDF contains one block per customer: name, QR code, activation code, package info. Store PDF URL or make available for partner download.

5. Supabase Edge Function Architecture

The entire pipeline above maps to Supabase Edge Functions (Deno/TypeScript). Currently your project has zero deployed functions — this is the build plan.

Function Name	Trigger	Responsibility
process-group-order	HTTP POST (from frontend)	Receives group_order_id. Runs Stage 0 → 1 → 2. Provisions all eSIMs. Returns summary.
dispatch-delivery	HTTP POST (called by process-group-order OR manually)	Runs Stage 3 → 4 → 5. Dispatches SMS/email/manual for a given group_order_id.
retry-failed-delivery	HTTP POST (manual trigger or cron)	Re-attempts delivery for orders with delivery_status = failed, attempt_count < 3.
get-esim-landing	HTTP GET (public, no auth)	Receives token. Returns eSIM data for the landing page. Rate limited.
sync-esim-usage	Cron (every 6–12h)	Polls eSIMAccess for data usage on all active eSIMs. Updates orders.order_usage, total_volume.
click-webhook	[PLACEHOLDER] HTTP POST	Will handle Click.uz Prepare/Complete callbacks when payment is integrated. Sets group_orders.status = 'paid'.

Why split process-group-order and dispatch-delivery?

Separation of concerns: provisioning (eSIMAccess API) and delivery (Eskiz/email) are independent failure domains.

Retry safety: if delivery fails for some customers after all eSIMs are provisioned, you can re-run dispatch-delivery without re-calling eSIMAccess.

Timeout management: Supabase Edge Functions have a ~150s wall-clock limit. Splitting the work prevents timeouts on large groups.

For small groups (< 10): process-group-order can call dispatch-delivery inline synchronously.

For large groups (> 10): process-group-order returns after provisioning, dispatch-delivery is called asynchronously.

6. What to Log, Flag, and Monitor

6.1 payment_audit_log Events

Every significant state change and API interaction must be written to payment_audit_log. This is the source of truth for debugging and reconciliation.

BULK_ORDER_STARTED	Stage 0 complete, pipeline beginning. Include member count, package_code.
ESIM_PROVISIONED	One eSIM provisioned successfully. Include iccid, member_id, order_id.
ESIM_PROVISION_FAILED	One eSIM provisioning failed. Include member_id, error from eSIMAccess, attempt number.
PROVISIONING_COMPLETE	Loop done. Include succeeded/failed counts, final group status.
DELIVERY_SENT	One delivery dispatched. Include method, recipient (masked), provider_message_id.
DELIVERY_FAILED	One delivery failed. Include method, reason, attempt_count.
BULK_ORDER_COMPLETED	Full pipeline done. Include final delivery summary.
BULK_ORDER_PARTIAL	Pipeline done with some failures. Include details for manual resolution.
MANUAL_PDF_GENERATED	PDF generated for manual delivery orders. Include download reference.

6.2 What to Flag for Attention

- Any group_order stuck in 'api_ordering' for > 10 minutes — likely crashed mid-loop
- Any order with order_status = 'FAILED' — needs manual resolution or refund
- Any delivery with attempt_count >= 3 — permanent failure, partner must be notified
- Eskiz balance below threshold — check before dispatching large SMS batches
- eSIMAccess API returning 4xx/5xx — may indicate auth token expiry or quota exceeded

6.3 Pre-dispatch Checks

Before Stage 4 (delivery), the service must verify:

- Eskiz: GET /api/user/get-limit — check balance > 0. If balance is zero, skip SMS, mark all SMS orders as 'DELIVERY_BLOCKED_NO_BALANCE', notify admin.
- Email provider: verify API key is valid (can do a test ping or just proceed and catch errors)
- Token uniqueness: verify no collision in orders.short_url before inserting
- Delivery contact validity: phone must be 12 digits (998XXXXXXXXX format for UZ). Email must contain @.

7. Failure Scenarios & How the Service Handles Them

Scenario	Detected By	Response
eSIMAccess API fails for 1 member	Step 1.8 — API error response	Log FAILED order row, continue loop. Final status = 'partial'. Partner notified. Admin flagged.
eSIMAccess API fails for ALL members	Step 2.2 — succeeded = 0	group_orders.status = 'failed'. Refund flag set. Admin notified. No delivery attempted.
Function crashes mid-loop	Step 0.2 idempotency check on restart	On re-invoke, already-provisioned orders are skipped. Loop continues from first non-provisioned member.
Eskiz SMS fails (invalid number)	Step 4.6 — Eskiz error response	Log failed delivery. eSIM is still provisioned. Mark for partner attention. Partner can resend or download QR.
Eskiz has zero balance	Step — pre-dispatch balance check	All SMS orders marked DELIVERY_BLOCKED_NO_BALANCE. Admin alerted. Partner can download manual QR PDF.
Token collision (extremely rare)	Step 1.5 — uniqueness query	Regenerate token and retry. Max 3 attempts. If all collide (impossible in practice), log and abort that member.
Partner has invalid delivery data	Step 0.4 — validation	Flag member as DELIVERY_BLOCKED. eSIM is still provisioned. Partner can update contact and trigger resend.
Function timeout (large group)	Supabase 150s limit	process-group-order provisions up to N members within time budget, sets status = 'api_ordering'. A follow-up retry call resumes.

8. The Tokenized eSIM Landing Page

8.1 URL Structure

onesim.uz/e/{token} — where token is 8 URL-safe characters generated by nanoid.

The page is public (no login required). Anyone with the link can view it. Security is through obscurity — the token space (62^8 = 218 trillion combinations) makes guessing infeasible.

8.2 What the Page Displays

- QR code — rendered client-side using qr_code_data or activation_code from the orders row
- LPA activation string — for manual eSIM installation (copy button)
- Package summary — country flag, data amount, validity days
- Partner/tour operator name — from partners.company_name (shows 'Powered by ONESIM' if partner allows)
- 3-step installation guide — illustrated, mobile-optimized
- ONESIM support link — Telegram/WhatsApp for help
- Expiry notice — if the link has expired, show a clean message with support contact

8.3 Security & Expiry

- Token expiry: travel_end_date + 60 days (set at provisioning time)
- After expiry: page shows 'This eSIM link has expired. Contact your operator.' — support link visible
- The eSIM itself is not affected by link expiry — only the page. Data plan validity is separate.
- View tracking: on first load, write first_viewed_at to esim_tokens (or orders metadata). Lets partner see who has/hasn't opened their link.
- Rate limit the endpoint: max 30 requests/minute per IP to prevent scraping

8.4 SMS Message Template

This is the full SMS text sent to the customer:

UZ destinations:

"{TourOpName}: eSIM tayyor! {Country} {DataGB}GB. Ochish: onesim.uz/e/{token}"

Example:

"Dubay Safari: eSIM tayyor! Dubai 5GB. Ochish: onesim.uz/e/aB3kR9xQ"

Approx length:

~65 characters = 1 SMS part. Cheap. ✓

9. Recommended Build Order

Build in this sequence to have a testable, deployable service as fast as possible:

4. Create esim_tokens table — simple, unblocks everything that follows
5. Build get-esim-landing Edge Function — public GET endpoint. Test with a manually inserted token.
6. Build the landing page at onesim.uz/e/[token] — static page reading from get-esim-landing.

7. Build process-group-order Edge Function — Stage 0 + Stage 1 only (provision loop). Test with a real group of 1 member against eSIMAccess sandbox.
8. Add Stage 2 (evaluation) + partner_earnings writes to process-group-order.
9. Build dispatch-delivery Edge Function — SMS branch first (Eskiz send-batch). Test end-to-end: provision → token → SMS → landing page.
10. Add Email branch to dispatch-delivery.
11. Add Manual branch (PDF generation or just flag orders for partner download).
12. Build retry-failed-delivery function.
13. [Later] Wire click-webhook → sets status = 'paid' → triggers process-group-order.

End of Document

ONESIM Bulk Order Service v1.0 | ONETECH PRO LLC | March 2026